M.Tech (VLSI Design) - Computational Project Report

Design and Fault-Injection Evaluation of a Self-Checking 4-bit ALU in Verilog

Inverse-operation arithmetic checking, parity prediction and a sharing-free bit-sliced datapath, compared against duplication with comparison

Submitted by: <Student name> (Roll No. <...>)
Specialization: VLSI / Digital Design
Guide: <Guide name>
<Department>, <Institute>

Tools: Icarus Verilog 12.0 · Yosys 0.33 with ABC · Verilator 5.020 · Python 3

Abstract

A 4-bit, 16-operation ALU (6 arithmetic, 4 logic, 6 shift/rotate operations with C, V, Z, N flags) is designed in synthesizable Verilog-2005 with three selectable adder architectures (ripple-carry, carry-lookahead, Kogge-Stone). The ALU is made self-checking with a concurrent error-detection (CED) checker that sees only the ALU inputs and outputs. Two arithmetic checks (residue-modulo-3 code and an inverse-operation check) and two logic checks (diverse duplication and parity prediction) are implemented, giving four checker configurations, and a bit-sliced, sharing-free datapath is proposed so that parity prediction becomes fault-secure after logic synthesis.

All 9 design variants × 3 adders were verified exhaustively (8192 input vectors each) at RTL and, after Yosys synthesis, at gate level (54 netlists). A bit-parallel fault simulator written for this project injected every single stuck-at fault into every netlist and simulated all 8192 vectors per fault; it was cross-validated against Icarus Verilog on all fault-free netlists and on 324 randomly chosen faulty netlists with exact agreement.

Main result: **the proposed configuration (inverse-operation + parity prediction + bit-sliced datapath) is fault-secure for 100% of internal single stuck-at faults**, the same as duplication with comparison (DMR), whereas the same checker on a conventionally synthesized datapath leaves 12.4% of effective internal faults able to produce silent errors, and the residue-3 variants leave 4.3–16.8%. At this small width the proposed design is not cheaper than DMR: 293 vs 289 generic gates (+1.4%) and an estimated 1920 vs 1740 transistors (+10.3%), with a deeper logic path (27 vs 20 levels). Its advantage is that, unlike DMR, the checker is an independent specification of the ALU and therefore also flags design errors: in a mutation experiment it raised an alarm for 8 of 9 injected datapath bugs on every corrupted vector (and on two-thirds of the vectors for the ninth), while DMR flagged none of them.

Contents

(Right-click → Update Field in Word to refresh page numbers.)

1. Introduction

The arithmetic-logic unit is the most heavily exercised block of any processor datapath. In safety-related and long-life systems (automotive, avionics, space, medical), a permanent or transient hardware fault in the ALU that silently produces a wrong result is the most dangerous failure mode, because software has no way to notice it. Concurrent error detection (CED) adds hardware that checks every result while the circuit operates normally and raises an error flag when the result is inconsistent with the inputs.

The classical CED solution is duplication with comparison (DMR). It is simple and catches every single fault inside one copy, but it more than doubles the area and it is blind to common-mode failures - most importantly, a design error is present in both copies and is never flagged. Code-based checkers (parity prediction, residue codes) are cheaper on wide datapaths and check the circuit against an independent formulation, but their guarantees rest on assumptions (e.g. a single fault flips only one output bit) that logic synthesis can silently break by sharing gates between bits.

1.1 Objectives

1. Design a complete, synthesizable 4-bit ALU in Verilog with 16 operations, 4 flags and three adder architectures.
2. Add a configurable self-checking circuit (residue-3 or inverse-operation for arithmetic; duplication or parity prediction for logic).
3. Verify every configuration exhaustively at RTL and gate level, and measure the quality of the testbench itself (mutation testing).
4. Synthesize every configuration with Yosys/ABC and compare area and logic depth against an unprotected ALU and a DMR ALU.
5. Measure the real fault-detection capability of each synthesized netlist with an exhaustive stuck-at fault-injection campaign, and identify why faults escape.

1.2 Contributions of this project

- **Hybrid, configurable checker** that observes only ALU inputs and outputs and has its own opcode decoder (alu4_checker.v).
- **Bit-sliced, sharing-free datapath** (alu4_datapath_bs.v) in which every output bit is computed by its own slice, kept as a separate hierarchy during synthesis, so that a single internal fault can corrupt at most one output bit of a non-arithmetic operation.
- **Exhaustive, bit-parallel fault simulator** (scripts/gatesim.py) that evaluates all 8192 input vectors of a gate netlist in one pass using 8192-bit integers, injected 40,588 faults across 54 netlists in about 10 seconds, and was cross-validated against Icarus Verilog.
- **Quantitative comparison** of 9 designs × 3 adders × 2 synthesis flows, including the negative results (residue-3 aliasing, parity escapes caused by logic sharing, area parity with DMR at 4 bits).

2. Background

2.1 Adder architectures

A ripple-carry adder (RCA) chains full adders; its carry delay grows linearly with width. A carry-lookahead adder (CLA) computes every carry directly from generate g = x·y and propagate p = x⊕y signals, e.g. c2 = g1 + p1g0 + p1p0c0, giving constant depth at the cost of wide AND-OR terms [3]. The Kogge-Stone (KS) adder is a parallel-prefix adder that combines (G, P) pairs over spans 1, 2, 4, … in log2(n) levels with minimal fan-out [4]. For 4 bits the KS tree has two prefix levels.

2.2 Concurrent error detection

- **Duplication with comparison (DMR):** two identical copies and an equality comparator. Detects any single fault in one copy; misses common-mode faults (design errors, faults in shared inputs).
- **Parity prediction:** the parity of the result is predicted from the parities of the operands and compared with the parity of the actual result [1, 6]. It detects all odd-weight errors, so it is fault-secure only if every single fault can change at most one output bit.
- **Residue code mod 3:** since a+y+ci = 16c + r, the residues must satisfy |a|3 + |y|3 + ci ≡ |r|3 + c (mod 3) because 16 ≡ 1 (mod 3) [2]. Any error of value ±2^k is detected, but errors whose value is a multiple of 3 alias.
- **Inverse operation:** the checker undoes the operation. For the adder, a + y + ci = 16c + r is equivalent to r + ~y + ~ci = {~c, a}, which the checker evaluates with its own adder. Any change of {c, r} changes the left-hand side, so every arithmetic error is detected.

A circuit is *fault-secure* for a fault set if, for every fault in the set and every input, the output is either correct or the error signal is raised [5]. This report measures exactly that property, per fault, over the complete input space.

3. Design of the ALU

3.1 Specification

Table 1: Instruction set (Z = (R == 0), N = R[3] for all operations)

op	Mnemonic	Result R	C flag	V flag
0	ADD	A + B	carry	yes
1	ADC	A + B + Cin	carry	yes
2	SUB	A − B	1 = no borrow	yes
3	SBB	A − B − Cin	1 = no borrow	yes
4	INC	A + 1	carry	yes
5	DEC	A − 1	1 = no borrow	yes
6	AND	A & B	0	0
7	OR	A | B	0	0
8	XOR	A ^ B	0	0
9	NOR	~(A | B)	0	0
A	NOT	~A	0	0
B	SHL	A << 1	A[3]	0
C	SHR	A >> 1 (logical)	A[0]	0
D	ASR	A >>> 1 (arithmetic)	A[0]	0
E	ROL	{A[2:0], A[3]}	A[3]	0
F	ROR	{A[0], A[3:1]}	A[0]	0

Ports of every core: a[3:0], b[3:0], op[3:0], cin → r[3:0], c, v, z, n, err, err_vec[3:0]. err_vec identifies which check failed: [0] arithmetic, [1] logic (AND/OR/NOR), [2] parity class (XOR/NOT/shift/rotate), [3] Z/N flags.

3.2 RTL modules

Table 2: RTL source files (rtl/)

File	Content
adder4.v	adder4_rca, adder4_cla, adder4_ks and the wrapper adder4 (parameter ADDER_ARCH)
alu4_datapath.v	Conventional datapath: operand conditioning, adder, logic, shifter, output case-mux, flags
alu4_datapath_bs.v	Proposed bit-sliced datapath: alu4_rslice ×4, alu4_cslice, alu4_vslice, shared adder
alu4_checker.v	CED checker; parameters ARITH_CHK (0 residue-3, 1 inverse) and LOGIC_CHK (0 duplication, 1 parity)
alu4_sc_core.v	Self-checking core = datapath (DP_STYLE) + checker
alu4_dmr_core.v	Reference: two datapaths + 8-bit equality comparator
alu4_base_core.v	Reference: unprotected datapath, err tied to 0
alu4_sc_top.v	Registered wrapper: input/output registers, err_now, sticky error flag, 8-bit error counter

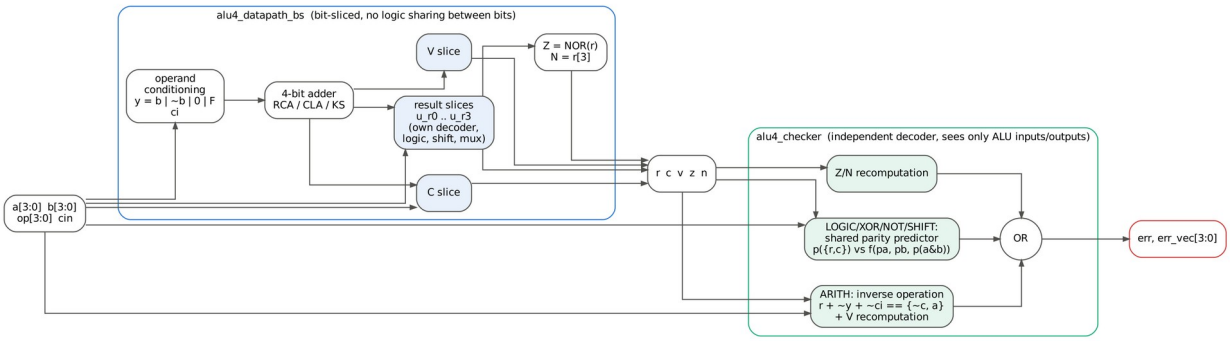


Figure 1: Architecture of the proposed self-checking ALU (alu4_sc_core with DP_STYLE = 1)

3.3 Datapath

The adder receives x = a and a conditioned operand y with carry-in ci: ADD/ADC use y = b, SUB/SBB use y = ~b with ci = 1 or ~cin (two's complement), INC uses y = 0, ci = 1 and DEC uses y = 1111, ci = 0. The overflow flag is V = (a3 ⊕ y3)·(s3 ⊕ a3), which is exact also with a carry-in. With this convention C = 1 after a subtraction means "no borrow", as in ARM processors.

Bit-sliced datapath. In alu4_datapath_bs every result bit r[i] is produced by its own module alu4_rslice #(BIT=i) containing a private opcode decoder, the bitwise logic for that bit, the shifter connections (a[i−1], a[i+1], a[3]) and the output multiplexer. C and V have their own slices; Z is the NOR of r and N is the wire r[3]. Only the operand conditioning and the adder are shared between bits, and their outputs are selected only by arithmetic opcodes. During synthesis each slice is kept as its own hierarchy (Yosys keep_hierarchy), so ABC cannot merge, for example, the four copies of the "op == AND" decoder into one shared gate.

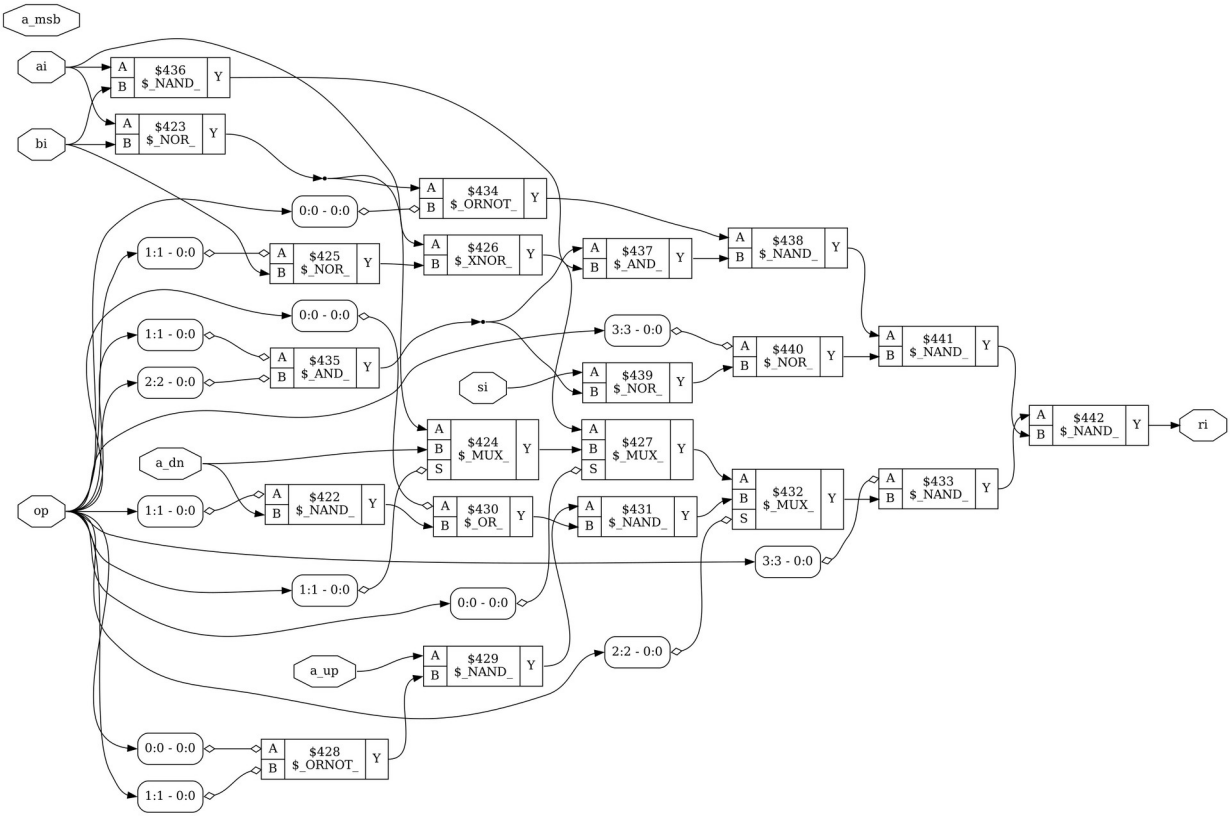


Figure 2: Gate-level schematic of one result slice (alu4_rslice, BIT = 1) produced by Yosys 'show' after ABC mapping

3.4 Checker

The checker decodes the opcode itself and evaluates one rule per class:

- **Arithmetic, residue-3 (ARITH_CHK = 0):** residues are formed with $|x|3 = |x[1:0]|3 + |x[3:2]|3$ (since $4 \equiv 1 \bmod 3$) and a mod-3 adder; the check is $|a|3 + |y|3 + ci \equiv |r|3 + c$.
- **Arithmetic, inverse (ARITH_CHK = 1):** inv = r + ~y + ~ci (5 bits) must equal {~c, a}. If c = 0 then r = a+y+ci and inv = a + 16; if c = 1 then inv = a. In both cases V is recomputed as (a3 ⊙ y3)(r3 ⊕ a3) from the checker's own y.
- **Logic, duplication (LOGIC_CHK = 0):** a&b, a|b, ~(a|b) are recomputed in De Morgan form (~(~a|~b) etc.) and compared bit by bit.
- **Logic, parity (LOGIC_CHK = 1):** p(a&b) = ⊕(a&b); since a|b = a ⊕ b ⊕ (a&b), p(a|b) = pa ⊕ pb ⊕ p(a&b); NOR inverts 4 bits so its parity equals p(a|b).
- **Parity class:** XOR → pa⊕pb; NOT → pa (4 inversions); SHL/SHR → p(r)⊕c = pa (the bit shifted out leaves through C); ASR → p(r)⊕c⊕a3 = pa (a3 is duplicated); ROL/ROR → p(r) = pa and C must equal the wrapped bit. One shared parity comparator serves all parity-checked opcodes.
- **Always:** Z is recomputed as NOR(r), N must equal r[3]; C and V must be 0 where the operation does not define them.

4. Functional verification

4.1 Exhaustive testbench

tb/tb_alu4_exhaustive.v applies all 2^13 = 8192 combinations of {op, a, b, cin} and compares r, c, v, z, n with a behavioural reference model (tb/alu4_ref.vh). The model is written with integer arithmetic - for example SUB's carry is computed as (a ≥ b) and V as "signed result outside [−8, 7]" - so it shares no structure with the RTL adder or operand mux. The testbench also requires err = 0 on every vector (no false alarms). The same testbench runs on RTL and, with -DGATE, on synthesized netlists.

Table 3: Verification summary (logs in sim/logs/)

Simulation	Configurations	Vectors each	Result
RTL exhaustive (Icarus)	18 SC + 3 DMR	8192	21 / 21 PASS
Gate-level exhaustive (Icarus)	54 netlists	8192	54 / 54 PASS
Verilator -Wall lint	36 module/parameter sets	-	0 warnings
RTL fault-injection demo	7 forced faults	192	all corrupted words flagged
Clocked top-level test	alu4_sc_top	396 checked ops	PASS (3 phases)

4.2 Testbench quality: mutation testing

A testbench that never fails proves little. scripts/mutation_test.py inserts one deliberate bug at a time into a copy of the RTL and re-runs the exhaustive testbench, which must report FAIL. All 14 mutants were killed. The same experiment exposes an important difference between the CED schemes: for every datapath bug the checker of the self-checking core raised err at run time, whereas a DMR core (whose two copies contain the same bug) never did.

Table 4: Mutation test results (vectors out of 8192; '-' = not applicable)

Injected bug	File	Wrong outputs	SC err raised	DMR err raised	TB
SBB uses wrong borrow polarity	alu4_datapath.v	512	512	0	KILLED
overflow uses B instead of adder operand	alu4_datapath.v	576	576	0	KILLED
ASR does not sign-extend	alu4_datapath.v	256	256	0	KILLED
ROR carry from wrong bit	alu4_datapath.v	256	256	0	KILLED
DEC subtracts 2 instead of 1	alu4_datapath.v	512	512	0	KILLED
bit-sliced: ROR uses wrong neighbour	alu4_datapath_bs.v	384	256	-	KILLED
bit-sliced: ROL carry missing	alu4_datapath_bs.v	256	256	-	KILLED
CLA c3 missing p[2] term	adder4.v	64	64	0	KILLED
KS prefix wired to wrong span	adder4.v	192	192	0	KILLED
checker[RES3]: residue of B used instead of adder operand	alu4_checker.v	0	1280	-	KILLED
checker[INV]: inverse carry-in not complemented	alu4_checker.v	0	3072	-	KILLED
checker[PAR]: OR parity misses AND term	alu4_checker.v	0	480	-	KILLED
checker: ASR parity rule wrong	alu4_checker.v	0	256	-	KILLED

Injected bug	File	Wrong outputs	SC err raised	DMR err raised	TB
checker: N recomputed from wrong bit	alu4_checker.v	0	3648	-	KILLED

Note that for the mutant "ROR uses wrong neighbour" the self-checking core flags only 256 of 384 wrong vectors: the bug permutes result bits, which leaves the parity unchanged on some inputs. Design-error detection by the checker is therefore a strong but not a guaranteed property.

4.3 RTL fault-injection demonstration

tb/tb_alu4_fault_demo.v runs a faulty and a golden copy of the proposed core side by side and uses Verilog force/release to inject one stuck-at fault per phase (CLA internal carry c2, adder operand bit y[1], result bits r[1] and r[2], flags c and z). In every phase every corrupted output word was flagged by err, and after release the design returned to error-free operation.

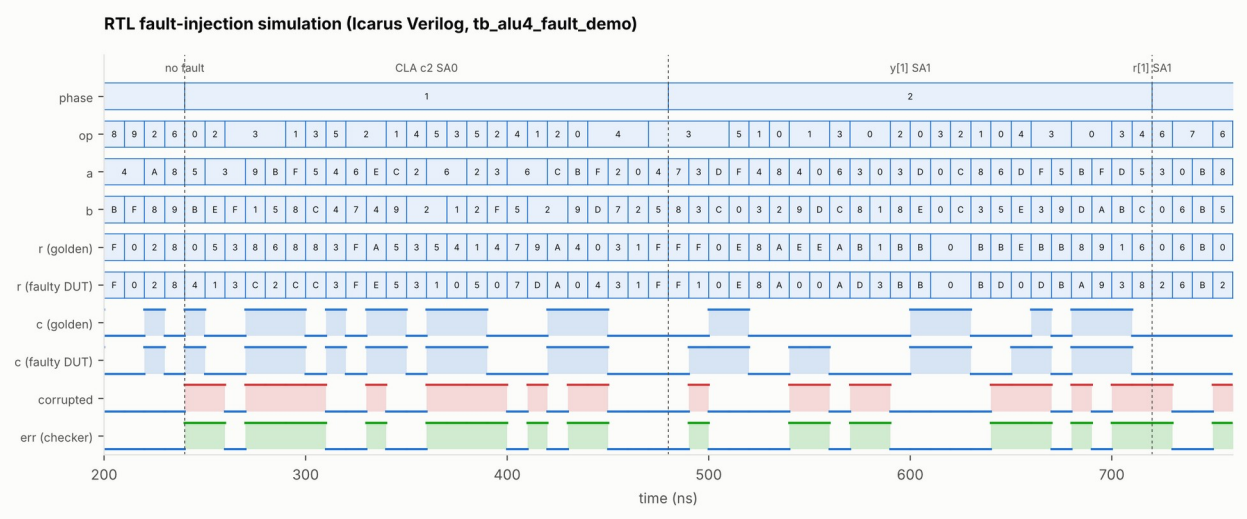


Figure 3: Waveform of the RTL fault-injection run (200-760 ns): CLA carry c2 stuck-at-0 and operand bit y[1] stuck-at-1. 'corrupted' marks vectors where the DUT differs from the golden copy; err rises on exactly those vectors.

Table 5: RTL fault-injection demo (sim/logs/rtl_fault_demo.log)

Phase	Fault	Vectors	Corrupted	Detected	Missed	False alarm
0	no fault	24	0	0	0	0
1	CLA c2 stuck-at-0	24	14	14	0	0
2	y[1] stuck-at-1	24	11	11	0	0
3	r[1] stuck-at-1	24	16	16	0	0
4	r[2] stuck-at-0	24	16	16	0	0
5	c stuck-at-1	24	18	18	0	0
6	z stuck-at-0	24	2	2	0	0
7	released	24	0	0	0	0

5. Synthesis and fault-injection methodology

5.1 Logic synthesis

scripts/run_synth.py generates and runs one Yosys script per variant (9 designs × 3 adders × 2 flows = 54 netlists). In both flows the datapath(s), the checker and, in the bit-sliced design, every slice are marked keep_hierarchy, so the optimizer cannot share logic between a function and its checker or merge the two DMR copies; the netlist is flattened only after mapping.

- **Structure-preserving flow:** `synth -flatten -noabc` - RTL operators are mapped directly to generic gates; the adder architecture remains visible.
- **Optimised flow:** `synth -flatten` followed by two further `abc -g AND,NAND,OR,NOR,XOR,XNOR,ANDNOT,ORNOT,MUX` passes - technology-independent multi-level optimisation.

No standard-cell library was used, so area is reported as the number of 2-input generic gates and as an estimated static-CMOS transistor count (NOT 2, NAND/NOR 4, AND/OR/ANDNOT/ORNOT 6, XOR/XNOR/MUX2 12), and speed as logic depth (gates on the longest input-to-output path). These are relative, technology-independent figures.

5.2 Fault model and campaign

Every net of a flattened netlist - each of the 13 primary-input bits and each gate output - is forced to stuck-at-0 and to stuck-at-1 (the stem fault list). For each fault all 8192 input vectors are simulated and compared with the fault-free netlist. With diff = vectors on which {r, c, v, z, n} is wrong and err = vectors on which the error flag is raised, every fault is classified as:

- **SDC** (silent data corruption) - some vector has diff ∧ ¬err;
- **DETECTED** - diff occurs, and on every such vector err = 1 (fault-secure);
- **ALARM_ONLY** - outputs never wrong, err raised on some vector (e.g. a checker fault);
- **SILENT** - no observable effect (redundant or masked logic).

The headline metric is the **fault-secure coverage** = (effective faults that never cause an SDC) / (effective faults). Faults on the 13 primary inputs are reported separately: an input that is wrong before it reaches the ALU is common-mode for every CED scheme including DMR (all 26 such faults are SDCs in every design), and in practice it is covered by parity on the operand bus. Metrics labelled "internal" exclude them.

5.3 Bit-parallel fault simulator and its validation

scripts/gatesim.py reads the Yosys JSON netlist, sorts the gates topologically and represents every net by one Python integer with 8192 bits - bit i is the value of the net for input vector i. A gate is then evaluated for all vectors with one bitwise operation (AND → &, XOR → ^, NOT → ^mask, MUX → (A & ~S) | (B & S)). A stuck-at fault is simulated by overriding one net with all-0 or all-1. The complete campaign of 40,588 faults over 54 netlists, each against 8192 vectors (≈332 million fault-vector simulations), takes about 10 s.

Because the conclusions depend on this tool, it was validated independently (scripts/crossval_iverilog.py, fault/crossval_report.txt):

- **Fault-free:** for all 54 netlists the Python outputs equal the Icarus Verilog gate-level vector dumps on all 13 output bits for all 8192 vectors.

- **Faulty:** for 6 random faults per netlist (324 faults) the fault was inserted into the JSON netlist, Yosys wrote a new Verilog netlist and Icarus simulated it; the numbers of corrupted, flagged and silently corrupted vectors matched the Python prediction exactly in all 324 cases.

6. Results

6.1 Adder architectures

Table 6: Effect of the adder architecture on the complete ALU

Adder (unprotected ALU)	Gates, struct.	Depth, struct.	Gates, ABC	Depth, ABC
Ripple-carry	215	20	137	16
Carry-lookahead	233	18	137	16
Kogge-Stone	221	18	137	16

With structure preserved, the CLA and KS adders reduce the ALU depth from 20 to 18 levels at the cost of 18 and 6 extra gates. After ABC optimisation the three architectures converge to essentially the same 137-gate, 16-level circuit: for a 4-bit adder the optimiser re-derives a near-optimal structure regardless of how it was written, and the critical path is dominated by opcode decoding and the 16-way result multiplexer rather than by the carry chain. The adder choice therefore matters little at this width, and it has almost no influence on the CED results below (after ABC the internal fault-secure coverage of every design differs by less than one percentage point between RCA, CLA and KS, and is identical for DMR and all inverse-operation designs).

6.2 Area and logic depth of the CED schemes

Table 7: Synthesis results (synth/out/*/stat.txt; synth/synthesis_summary.csv)

Design (CLA adder)	Gates (ABC)	Transistors est. (ABC)	Depth (ABC)	Gates (struct.)	Depth (struct.)
Base (no CED)	137	802	16	233	18
Base, bit-sliced	154	952	17	407	18
DMR (duplication)	289	1740	20	481	22
RES3 + DUP	329	2000	29	515	35
RES3 + PAR	313	1916	29	495	35
INV + DUP	292	1854	27	437	27
INV + PAR	276	1770	27	413	27
RES3 + PAR, bit-sliced	323	2042	29	662	35
INV + PAR, bit-sliced (proposed)	293	1920	27	587	27

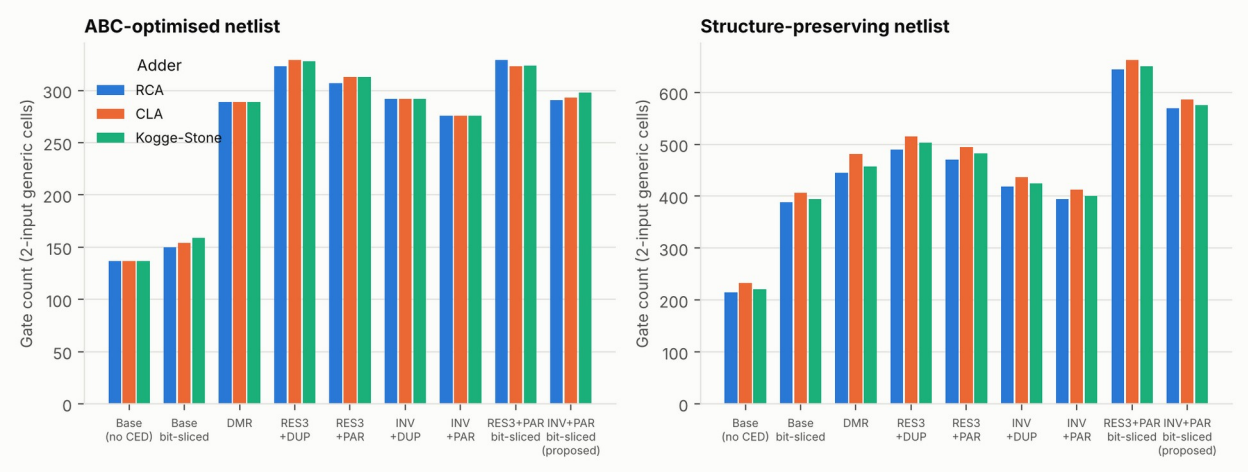


Figure 4: Gate count of every design for the three adders, after ABC optimisation (left) and with structure preserved (right)

The unprotected ALU needs 137 gates. DMR costs 289 gates (2 × 137 + a 15-gate comparator). The checkers cost between 139 and 192 gates, i.e. roughly one extra copy of the ALU: at a width of 4 bits a code-based checker is not cheaper than duplication, because the parity trees, the residue generators and the checker's own opcode decoder are comparable in size to the tiny datapath they check. The inverse-operation check is consistently smaller than residue-3 (no mod-3 generators are needed), and parity prediction is smaller than duplication for the logic operations. Making the datapath sharing-free costs 17 gates after ABC (137 → 154), which is modest because ABC still optimises inside each slice; the structure-preserving flow shows the raw cost (233 → 407). The checkers lengthen the critical path from 16 to 27-29 levels because err depends on the ALU result; DMR needs only 20.

6.3 Fault-injection results

Table 8: Exhaustive stuck-at fault-injection results, internal faults, ABC netlists unless stated (fault/results/summary.csv)

Design (CLA adder)	Effective internal faults	Faults with SDC	Fault-secure cov. %	Corrupted vectors flagged %	P(SDC) %	Fault-secure cov. %, struct.
Base (no CED)	274	274	0.00	0.00	25.580	0.00
DMR (duplication)	274	0	100.00	100.00	0.000	100.00
RES3 + DUP	280	42	85.00	95.79	0.456	79.50
RES3 + PAR	280	47	83.21	95.03	0.565	78.03
INV + DUP	274	29	89.42	97.83	0.261	86.21
INV + PAR	274	34	87.59	97.05	0.375	84.27
RES3 + PAR, bit-sliced	300	13	95.67	98.44	0.109	95.27
INV + PAR, bit-sliced (proposed)	308	0	100.00	100.00	0.000	100.00

P(SDC) is the probability that a randomly chosen internal fault and a randomly chosen operation together produce a silently wrong result. For the unprotected ALU it is about 26%; the proposed design and DMR reduce it to exactly zero for single stuck-at faults.

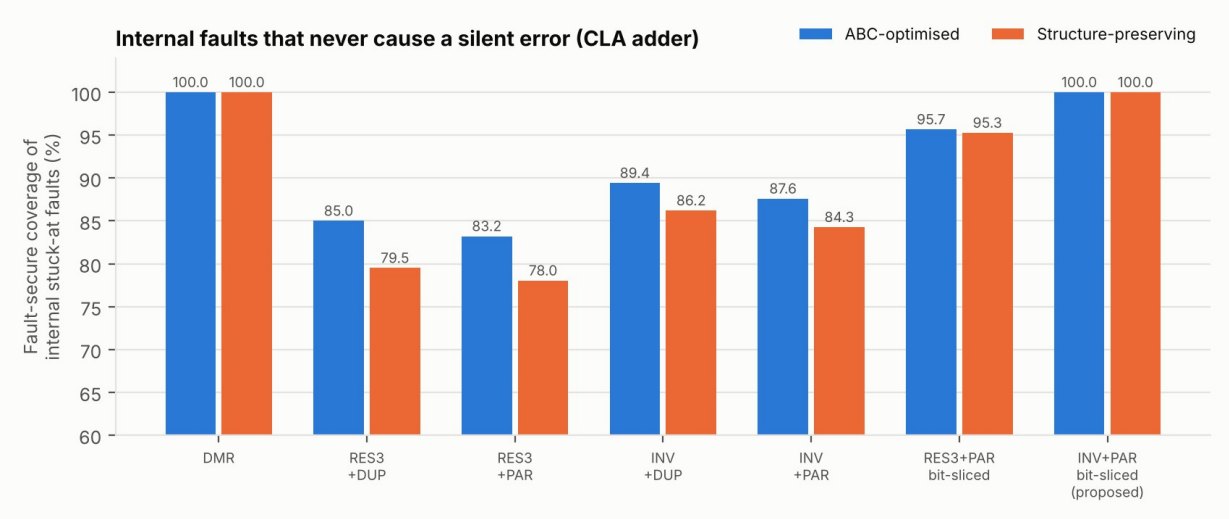


Figure 5: Fault-secure coverage of internal faults for every CED scheme (CLA adder)

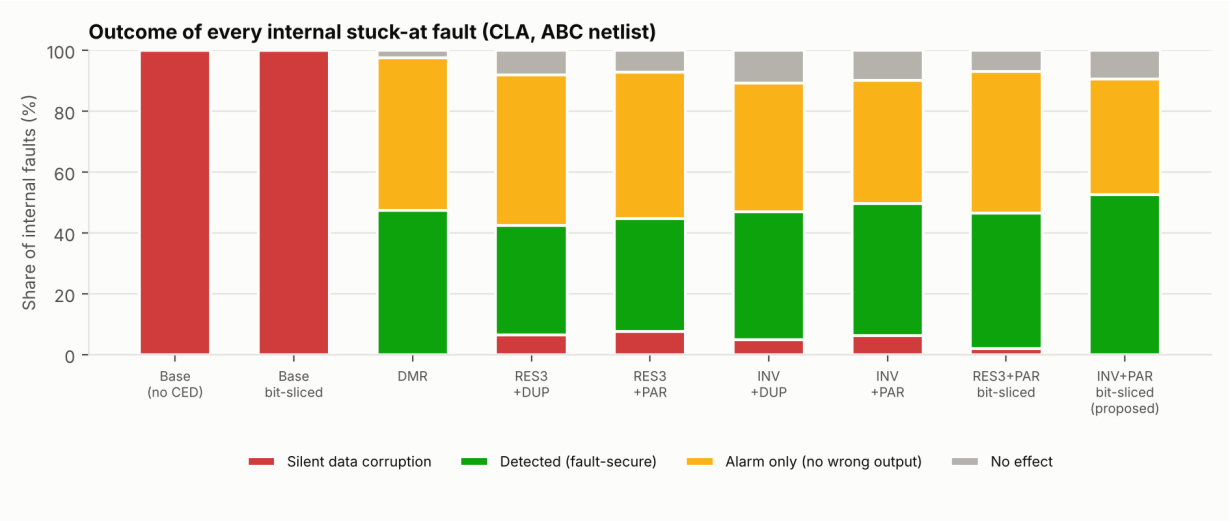


Figure 6: Classification of every internal stuck-at fault (CLA adder, ABC netlist)

6.4 Why faults escape - analysis of the SDC faults

Parity escapes from logic sharing. On the conventional datapath, the inverse + parity checker leaves 34 internal faults that can cause silent errors, all of them in the datapath and all of them on non-arithmetic opcodes (fault/results/sc_invpar_cla_opt_faults.csv). They sit on gates that ABC shares between result bits - typically a decoded select line or a common sub-expression of the 16-way multiplexer. Such a fault flips two result bits at once on some vectors; the parity of a double error is unchanged, so the error escapes. The bit-sliced datapath removes exactly these faults: with the same checker the count drops to 0.

Residue-3 aliasing. With the residue check, faults in the adder operand conditioning can change the adder operand by a multiple of 3 (for example y = b instead of ~b changes y by 15 − 2b, and with the carry-in also wrong the total error can be ≡ 0 mod 3). These faults remain even in the bit-sliced design (13 faults, 95.7% coverage). The inverse-operation check has no aliasing because it compares the full 5-bit value.

Duplication of the logic unit does not help on its own: INV + DUP still has 29 SDC faults because XOR, NOT and the shift/rotate operations are still parity-checked on a datapath with shared logic.

Checker faults never cause SDCs under the single-fault model (the datapath output is still correct); they appear as ALARM_ONLY (223 in the proposed design) or SILENT (55). The silent checker faults are latent: a later second fault in the datapath could then go undetected. The checker is therefore not totally self-checking; making it self-testing (e.g. with two-rail outputs) is listed as future work.

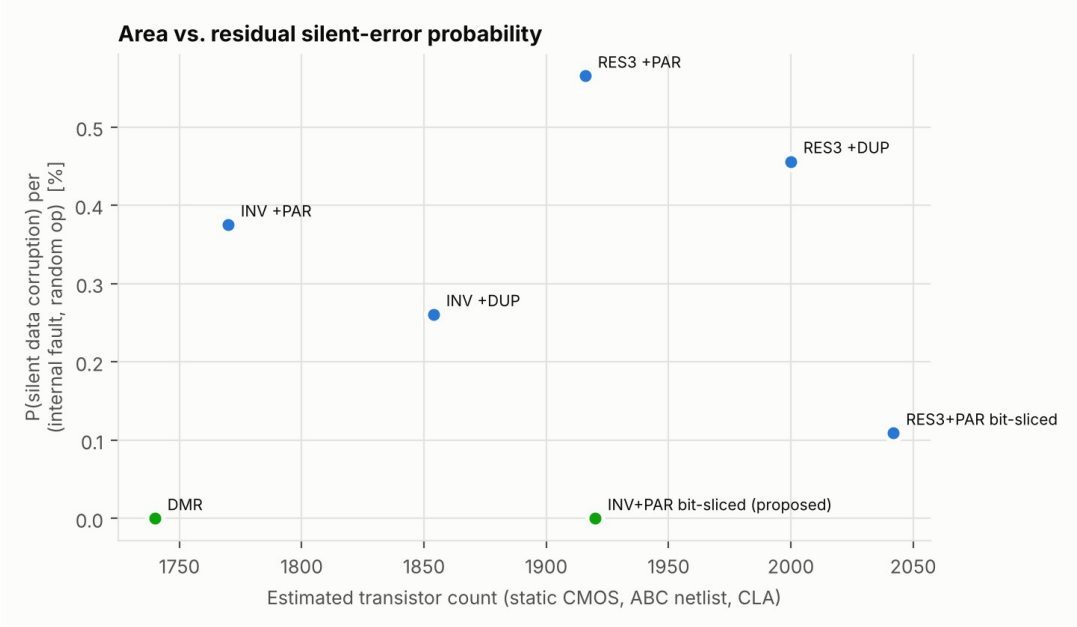


Figure 7: Area versus probability of silent data corruption (CLA adder, ABC netlists)

7. Discussion

The experiments give an honest picture of where each technique stands for a 4-bit ALU:

- **Coverage:** the proposed design reaches the same 100% internal fault-secure coverage as DMR. Code-based checking on an ordinarily synthesized datapath does not: 10-17% of the effective internal faults can still corrupt results silently.
- **Area and speed:** DMR is the cheaper and faster of the two fully fault-secure solutions at this width (289 vs 293 gates, 20 vs 27 levels). The checker is a fixed-size block per class of operation while the datapath it protects is very small; the balance is expected to shift towards code-based checking for wider datapaths, but that was not measured here and remains a hypothesis.
- **Independence:** the checker is a second, structurally different description of the ALU. It flagged the run-time effect of every injected design bug in the datapath (8 of 9 on every wrong vector, 1 on two-thirds of them) while DMR flagged none, and it reports which class of operation failed (err_vec).
- **Synthesis awareness:** the largest single effect in the study is not the choice of code but the synthesis structure. Any parity-based CED must constrain the synthesis tool (here with per-slice keep_hierarchy); otherwise its fault-secureness is lost without any visible change in function.

7.1 Limitations

- Area and delay are technology-independent estimates (generic gates, transistor counts, logic levels), not results of a standard-cell flow with a real liberty file; no power analysis was done.
- The fault model is single stuck-at on net stems. Branch (fan-out) faults, bridging faults, delay faults and transient (SEU/SET) faults were not simulated.
- The checker is not totally self-checking; latent checker faults are possible.
- Primary-input faults are undetectable by all on-chip CED schemes studied and must be covered upstream (e.g. operand-bus parity).

8. Conclusion and future work

A complete self-checking 4-bit ALU was designed in Verilog, verified exhaustively at RTL and gate level, synthesized in 54 configurations and evaluated with an exhaustive, independently validated stuck-at fault-injection campaign. Combining an inverse-operation check for arithmetic with parity prediction for all other operations, and restructuring the datapath into sharing-free bit slices, yields a design whose internal single stuck-at faults never produce a silent error - matching DMR - while additionally detecting most design errors that DMR cannot. At 4 bits this costs about the same gate count as DMR (+1.4%) and a longer critical path.

Future work: (1) parameterise the width (8/16/32 bits) and measure where the code-based checker becomes cheaper than duplication; (2) map to a real standard-cell library (e.g. an open PDK) for area, timing and power; (3) a two-rail, self-testing checker; (4) extend the campaign to transient faults and to branch faults; (5) pipeline the checker to remove it from the critical path.

References

[1] G. G. Langdon and C. K. Tang, "Concurrent error detection for group look-ahead binary adders," IBM Journal of Research and Development, vol. 14, no. 5, pp. 563-573, 1970.

[2] A. Avizienis, "Arithmetic error codes: cost and effectiveness studies for application in digital system design," IEEE Transactions on Computers, vol. C-20, no. 11, pp. 1322-1331, 1971.

[3] A. Weinberger and J. L. Smith, "A logic for high-speed addition," National Bureau of Standards Circular 591, pp. 3-12, 1958.

[4] P. M. Kogge and H. S. Stone, "A parallel algorithm for the efficient solution of a general class of recurrence equations," IEEE Transactions on Computers, vol. C-22, no. 8, pp. 786-793, 1973.

[5] P. K. Lala, Self-Checking and Fault-Tolerant Digital Design. San Francisco: Morgan Kaufmann, 2001.

[6] M. Nicolaidis, "Carry checking/parity prediction adders and ALUs," IEEE Transactions on VLSI Systems, vol. 11, no. 1, pp. 121-128, 2003.

[7] C. Wolf, "Yosys Open SYnthesis Suite," https://yosyshq.net/yosys/.

[8] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in Proc. CAV, LNCS 6174, pp. 24-40, 2010.

[9] S. Williams, "Icarus Verilog," https://steveicarus.github.io/iverilog/.

[10] W. Snyder, "Verilator," https://www.veripool.org/verilator/.

Appendix A. Project files and how to reproduce

```
alu4_selfcheck/
  rtl/        Verilog RTL (8 files, see Table 2)
  tb/         tb_alu4_exhaustive.v, tb_alu4_fault_demo.v, tb_alu4_top.v, alu4_ref.vh
  scripts/    gatesim.py         bit-parallel gate & fault simulator
              run_synth.py       Yosys flow for the 54 variants
              fault_campaign.py  exhaustive stuck-at campaign
              crossval_iverilog.py  validation against Icarus Verilog
              mutation_test.py   testbench mutation test
              make_figures.py    all figures of this report
  synth/out/<variant>/  synth.ys, yosys.log, stat.txt, netlist.v, netlist.json
  sim/        logs/ (all simulation logs), vectors/ (8192-vector dumps), waves/ (VCD)
  fault/      results/summary.csv, results/<variant>_faults.csv, crossval_report.txt
  docs/       this report, figures/
  run_all.sh  run_lint.sh  run_rtl_sims.sh  run_gate_sims.sh
```

The complete flow is reproduced with one command (about 2.5 minutes on a laptop):

```
$ ./run_all.sh
# requires: iverilog (12), yosys (0.33+), verilator (5), python3 + matplotlib, graphviz
```

Individual steps: ./run_lint.sh, ./run_rtl_sims.sh, python3 scripts/run_synth.py, ./run_gate_sims.sh, python3 scripts/fault_campaign.py, python3 scripts/crossval_iverilog.py. Waveforms (sim/waves/*.vcd) can be opened with GTKWave.